TRABAJO PRÁCTICO OBLIGATORIO

Programación Orientada a Objetos (POO)

Propuesta B — Agencia de Turismo

“ Rumbo Sur ”

ETAPA 1 · MODELADO Y APLICACIÓN DE PATRONES

Integrantes:

- Lopez Monea, Santiago (Legajo: 1213045)
- Rivas, Carlos Thomas (Legajo: 1208227)
- Zornada Alvarez Vega, Bryan (Legajo: 1210869)

Docente: León, Maria Angela

Fecha de entrega: 25 de octubre de 2026

1. Análisis del Enunciado

1.1. Identificación de actores y supuestos

Vendedor: Encargado de gestionar las ventas de la agencia. Registra clientes y realiza reservas para las distintas salidas, quedando asociado a cada reserva realizada.

Coordinador de operaciones: Responsable de organizar las salidas de los tours. Se encarga de emitir las listas de pasajeros y cerrar las salidas una vez finalizadas.

Administrador: Encargado de consultar y generar reportes sobre el funcionamiento de la agencia, incluyendo la ocupación de las salidas, las ventas por vendedor, los ingresos por tour y las cancelaciones.

Supuesto: Se considera que el sistema es utilizado internamente por el personal de la agencia. Por lo tanto, los clientes, pasajeros, guías y proveedores no se incluyen como actores, ya que el enunciado no especifica que interactúen directamente con el sistema.

1.2. Requerimientos funcionales

1. **Administrar tours:** Permitir registrar y gestionar los tours de la agencia, incluyendo sus datos, categorías, itinerarios, actividades y tarifas según la temporada.
2. **Gestionar proveedores:** Registrar los proveedores y los servicios que ofrecen, asociándolos a las actividades de los tours cuando corresponda.
3. **Programar salidas:** Permitir programar las salidas de cada tour, estableciendo las fechas, los cupos disponibles y su estado.
4. **Asignar guías y vehículos:** Asignar un guía y un vehículo a cada salida, verificando que cumplan con los requisitos necesarios y que se encuentren disponibles.
5. **Registrar clientes:** Permitir registrar y consultar los datos personales de los clientes de la agencia.
6. **Gestionar reservas:** Crear reservas para una salida determinada, asociándolas a un cliente, un vendedor y uno o más pasajeros.
7. **Controlar los cupos:** Verificar la disponibilidad de lugares antes de realizar una reserva y actualizar los cupos según las reservas y cancelaciones.
8. **Calcular precios:** Calcular el importe de cada reserva teniendo en cuenta la tarifa del tour, la temporada, el tipo de pasajero y las promociones aplicables.
9. **Contratar servicios adicionales:** Permitir agregar servicios adicionales para cada pasajero, como seguros de viaje, traslados o excursiones opcionales.
10. **Registrar pagos:** Registrar la seña y los pagos posteriores de una reserva, contemplando distintos medios de pago, cuotas y posibles recargos.

11. **Controlar el estado de las reservas:** Confirmar las reservas cuando se cumplan las condiciones de pago y anular aquellas que no respeten los plazos establecidos.
12. **Modificar reservas:** Permitir realizar modificaciones en las reservas, incluyendo el reemplazo de pasajeros cuando se cumplan las condiciones establecidas.
13. **Gestionar cancelaciones:** Registrar las cancelaciones de reservas y calcular los importes a devolver según la política de cancelación de la agencia.
14. **Emitir listas de pasajeros:** Generar las listas correspondientes a cada salida, incluyendo los datos personales y los contactos de emergencia de los pasajeros.
15. **Gestionar el desarrollo de las salidas:** Controlar los estados de las salidas y permitir su inicio y cierre, respetando las condiciones establecidas.
16. **Liquidar honorarios:** Calcular los honorarios correspondientes a los guías por las salidas finalizadas, considerando los días trabajados y las ventas de servicios adicionales.
17. **Generar reportes:** Permitir consultar y emitir reportes de ocupación por salida, ventas por vendedor, ingresos por tour y cancelaciones.
18. **Registrar ajustes:** Permitir corregir reservas pagadas mediante notas de ajuste, sin eliminar los registros originales.

1.3. Reglas de negocio

Tours y salidas

1. Los tours pueden ser de aventura, culturales o de naturaleza. Los de aventura deben especificar una edad mínima y un nivel de dificultad.
2. Cada tour debe contar con un itinerario compuesto por actividades organizadas según el día y el orden en que se realizan.
3. Las salidas pueden tener los siguientes estados: programada, confirmada, completa, en curso, finalizada o cancelada.
4. Una salida se confirma cuando alcanza el cupo mínimo de pasajeros. Si siete días antes de su inicio no lo alcanza, se cancela y se devuelve el 100 % de lo pagado.
5. Cuando una salida alcanza el cupo máximo, pasa al estado completa y no permite nuevas reservas.
6. Un guía no puede tener dos salidas superpuestas y solamente puede ser asignado a tours de las categorías para las que está habilitado.
7. La capacidad del vehículo asignado no puede ser inferior al cupo de salida.

Reservas y precios

8. Cada reserva pertenece a un cliente y a una salida, y debe incluir uno o más pasajeros. Cada pasajero ocupa un lugar del cupo, independientemente de la tarifa que pague.
9. La tarifa base de cada tour depende de la temporada, que puede ser baja, media o alta.

10. El precio de cada pasajero depende de su categoría. Los adultos pagan el 100 % de la tarifa, los menores de 12 años el 60 %, los menores de 3 años no pagan y los jubilados tienen un 10 % de descuento.
11. Las reservas realizadas con más de 60 días de anticipación reciben un descuento del 15 %. Las reservas de seis o más pasajeros reciben un descuento del 10 %.
12. Las promociones no son acumulables. Si una reserva cumple con las condiciones de ambas, se aplica la más beneficiosa para el cliente.
13. Los servicios adicionales se contratan por pasajero y su costo se suma al precio correspondiente. En los tours de aventura, el seguro de viaje es obligatorio.

Pagos

14. Para confirmar una reserva, el cliente debe abonar una seña del 30 % del importe total.
15. El saldo restante debe pagarse hasta diez días antes de la salida. Si no se completa el pago dentro del plazo, la reserva se anula y los lugares quedan nuevamente disponibles.
16. Los pagos pueden realizarse en cuotas y mediante efectivo, transferencia o tarjeta. Los pagos con tarjeta en cuotas tienen un recargo del 7 %.

Cancelaciones y modificaciones

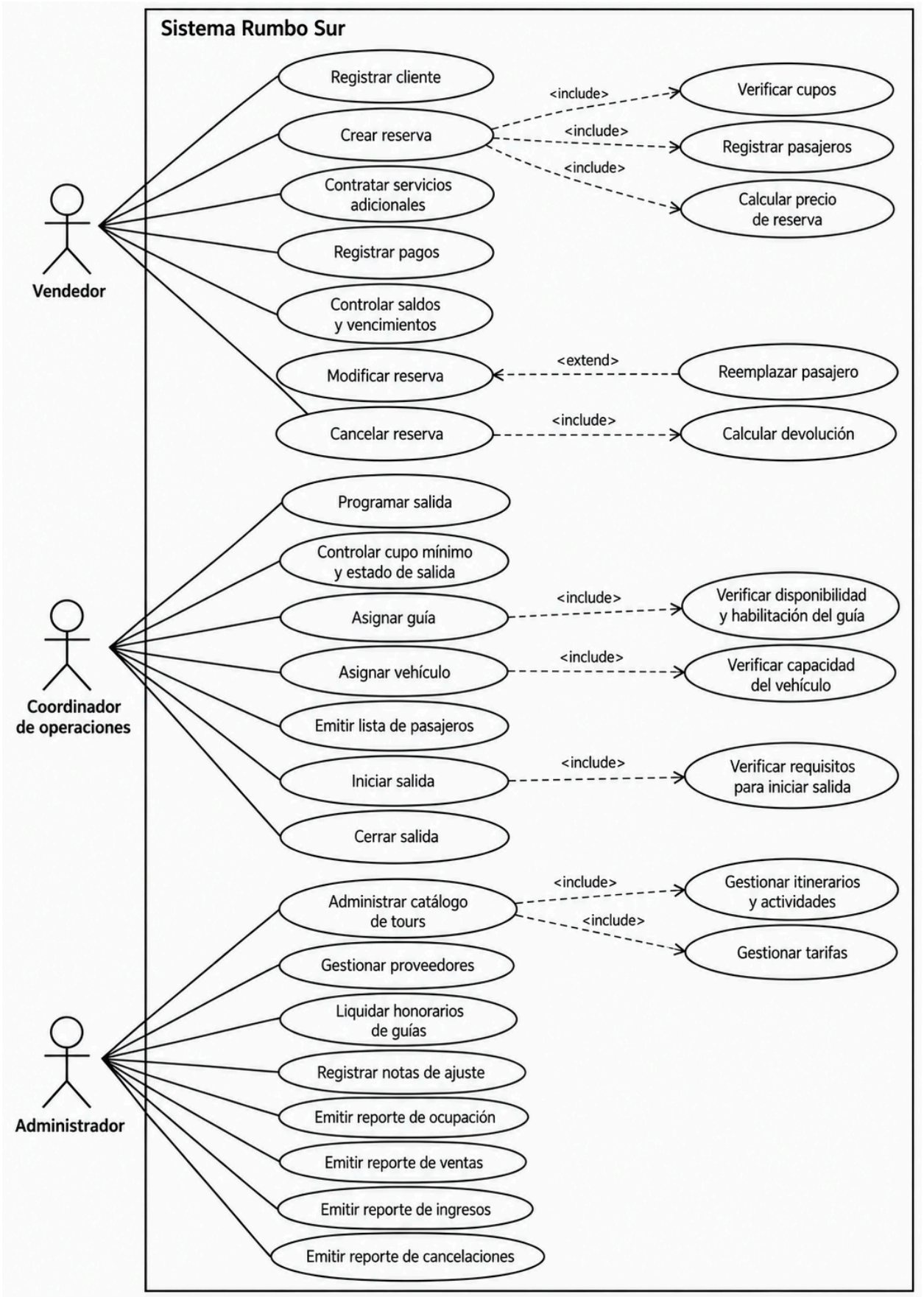
17. Las devoluciones por cancelación dependen de la anticipación con la que se realicen:
    - Hasta 30 días antes de la salida: devolución del 90 %.
    - Entre 29 y 15 días antes: devolución del 50 %.
    - Entre 14 y 7 días antes: devolución del 20 %.
    - Con menos de 7 días de anticipación: sin devolución.
18. Cuando una salida es cancelada por la agencia, corresponde devolver el 100 % del dinero abonado.
19. Se permite reemplazar un pasajero por otro hasta tres días antes de la salida, sin costo adicional, siempre que el nuevo pasajero cumpla con las condiciones del tour.
20. Las cancelaciones deben registrar la fecha, el motivo y el importe devuelto. Las devoluciones se realizan por el mismo medio de pago utilizado originalmente.

Operación y liquidaciones

21. Antes de iniciar una salida, el coordinador debe emitir la lista de pasajeros con sus documentos y contactos de emergencia. No se puede iniciar una salida sin esta lista ni sin un guía asignado.
22. Los guías cobran honorarios por cada salida finalizada, calculados mediante un monto fijo por día trabajado más un 5 % de las ventas de servicios adicionales de esa salida.
23. Las reservas pagadas no pueden eliminarse del sistema. En caso de errores o correcciones, se deben registrar notas de ajuste que permitan conservar la información original.

2. Diagrama de casos de uso

2.1. Diagrama de casos de uso



2.2. Descripción de los casos de uso

Vendedor

1. Registrar cliente

- **Actor:** Vendedor.
- **Precondición:** Contar con los datos personales del cliente.
- **Flujo principal:** El vendedor ingresa los datos del cliente y el sistema los registra.
- **Flujo alternativo:** No se especifica en el enunciado.

2. Crear reserva

- **Actor:** Vendedor.
- **Precondición:** Tener un cliente registrado y una salida disponible.
- **Flujo principal:** El vendedor selecciona la salida, registra los pasajeros, verifica los cupos disponibles y calcula el precio total de la reserva.
- **Flujo alternativo:** Si no hay suficientes lugares, no se permite realizar la reserva.

3. Registrar pasajeros

- **Actor:** Vendedor.
- **Precondición:** Estar realizando una reserva.
- **Flujo principal:** Se registran los datos personales de uno o más pasajeros, incluyendo documento, fecha de nacimiento, contacto de emergencia y restricciones alimentarias o de salud.
- **Flujo alternativo:** Si un pasajero no cumple con la edad mínima de un tour de aventura, no puede participar.

4. Verificar cupos

- **Actor:** Vendedor.
- **Precondición:** Tener una salida seleccionada y conocer la cantidad de pasajeros.
- **Flujo principal:** El sistema verifica que haya suficientes lugares disponibles para los pasajeros de la reserva.
- **Flujo alternativo:** Si se supera el cupo máximo de la salida, no se permite confirmar la reserva de lugares.

5. Calcular precio de reserva

- **Actor:** Vendedor.
- **Precondición:** Tener seleccionados el tour, la salida y los pasajeros.
- **Flujo principal:** El sistema calcula el precio considerando la tarifa por temporada, el tipo de pasajero, las promociones y los servicios adicionales.
- **Flujo alternativo:** Si corresponden las promociones por grupo y reserva anticipada, se aplica solamente la más favorable.

6. Contratar servicios adicionales

- **Actor:** Vendedor.
- **Precondición:** Tener una reserva en proceso o registrada.
- **Flujo principal:** Se seleccionan los servicios adicionales para cada pasajero y se incorporan sus importes al total de la reserva.
- **Flujo alternativo:** Si el tour es de aventura, se debe incluir obligatoriamente el seguro de viaje.

7. Registrar pagos

- **Actor:** Vendedor.
- **Precondición:** Tener una reserva registrada.
- **Flujo principal:** El vendedor registra los pagos realizados, indicando fecha, importe y medio de pago. La reserva queda confirmada al alcanzar la seña del 30 %.
- **Flujo alternativo:** Si el pago se realiza con tarjeta en cuotas, se aplica un recargo del 7 %.

8. Controlar saldos y vencimientos

- **Actor:** Vendedor.
- **Precondición:** Tener reservas con pagos pendientes.
- **Flujo principal:** El sistema verifica los saldos pendientes y los compara con la fecha límite de pago, establecida diez días antes de la salida.
- **Flujo alternativo:** Si el saldo no se cancela dentro del plazo establecido, la reserva se anula y los lugares vuelven a estar disponibles.

9. Modificar reserva

- **Actor:** Vendedor.
- **Precondición:** Tener una reserva registrada.
- **Flujo principal:** El vendedor selecciona una reserva y realiza la modificación solicitada, respetando las condiciones establecidas.
- **Flujo alternativo:** Si la modificación implica reemplazar un pasajero, se deben verificar los requisitos y plazos correspondientes.

10. Reemplazar pasajero

- **Actor:** Vendedor.
- **Precondición:** Tener una reserva registrada y encontrarse al menos tres días antes de la salida.
- **Flujo principal:** El vendedor registra al nuevo pasajero y verifica que cumpla con las condiciones del tour. El reemplazo se realiza sin cargo.
- **Flujo alternativo:** Si no se cumple el plazo o el nuevo pasajero no reúne los requisitos, no se permite el reemplazo.

11. Cancelar reserva

- **Actor:** Vendedor.
- **Precondición:** Tener una reserva registrada.
- **Flujo principal:** El vendedor selecciona la reserva, registra la fecha y el motivo de cancelación, y el sistema calcula el importe correspondiente a devolver.
- **Flujo alternativo:** Si la cancelación la realiza la agencia, corresponde devolver el 100 % de lo abonado.

12. Calcular devolución

- **Actor:** Vendedor.
- **Precondición:** Tener una reserva que se desea cancelar.
- **Flujo principal:** El sistema calcula el importe a devolver según la anticipación de la cancelación: 90 % con al menos 30 días, 50 % entre 29 y 15 días, 20 % entre 14 y 7 días, y ninguna devolución con menos de 7 días.
- **Flujo alternativo:** Si la cancelación es responsabilidad de la agencia, se devuelve el 100 % de lo pagado.

Coordinador de operaciones

13. Programar salida

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener un tour registrado.
- **Flujo principal:** El coordinador programa una salida indicando el tour, las fechas de inicio y fin y el cupo correspondiente.
- **Flujo alternativo:** No se especifica en el enunciado.

14. Controlar cupo mínimo y estado de salida

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener una salida programada.
- **Flujo principal:** El sistema verifica la cantidad de pasajeros reservados y confirma la salida cuando se alcanza el cupo mínimo.
- **Flujo alternativo:** Si siete días antes de la salida no se alcanza el mínimo requerido, se cancela y se devuelve el 100 % de lo pagado.

15. Asignar guía

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener una salida programada.
- **Flujo principal:** El coordinador selecciona un guía para la salida y el sistema verifica que se encuentre habilitado y disponible.
- **Flujo alternativo:** Si el guía no cumple las condiciones necesarias, no se permite su asignación.

16. Verificar disponibilidad y habilitación del guía

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener un guía seleccionado y una salida programada.
- **Flujo principal:** El sistema verifica que el guía esté habilitado para la categoría del tour y que no tenga otra salida superpuesta.
- **Flujo alternativo:** Si no está habilitado o presenta superposición de horarios, se rechaza su asignación.

17. Asignar vehículo

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener una salida programada.
- **Flujo principal:** El coordinador selecciona un vehículo para la salida y el sistema verifica que tenga capacidad suficiente.
- **Flujo alternativo:** Si el vehículo no cumple con la capacidad requerida, no se permite asignarlo.

18. Verificar capacidad del vehículo

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener un vehículo seleccionado y conocer el cupo de la salida.
- **Flujo principal:** El sistema compara la capacidad del vehículo con el cupo establecido para la salida.
- **Flujo alternativo:** Si la capacidad es inferior al cupo, se rechaza la asignación.

19. Emitir lista de pasajeros

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener una salida con pasajeros registrados.
- **Flujo principal:** El coordinador solicita la lista de pasajeros y el sistema la genera con los documentos y contactos de emergencia correspondientes.
- **Flujo alternativo:** No se especifica en el enunciado.

20. Iniciar salida

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener una salida preparada para comenzar.
- **Flujo principal:** El coordinador solicita el inicio de la salida y el sistema verifica las condiciones necesarias antes de actualizar su estado a en curso.
- **Flujo alternativo:** Si no se cumplen los requisitos de inicio, el sistema impide comenzar la salida.

21. Verificar requisitos para iniciar salida

- **Actor:** Coordinador de operaciones.
- **Precondición:** Tener una salida que se desea iniciar.
- **Flujo principal:** El sistema comprueba que la salida tenga un guía asignado y que se haya emitido la lista de pasajeros.
- **Flujo alternativo:** Si falta alguna de estas condiciones, no se permite iniciar la salida.

22. Cerrar salida

- **Actor:** Coordinador de operaciones.
- **Precondición:** Que la salida haya terminado.
- **Flujo principal:** El coordinador cierra la salida y el sistema actualiza su estado a finalizada, permitiendo posteriormente la liquidación de honorarios del guía.
- **Flujo alternativo:** No se especifica en el enunciado.

<u>Administrador</u>

23. Administrar catálogo de tours

- **Actor:** Administrador.
- **Precondición:** Contar con la información necesaria del tour.
- **Flujo principal:** El administrador registra y gestiona los tours de la agencia, incluyendo código, nombre, descripción, duración, categoría y cupos.
- **Flujo alternativo:** Si se trata de un tour de aventura, deben especificarse también la edad mínima y el nivel de dificultad.

24. Gestionar itinerarios y actividades

- **Actor:** Administrador.
- **Precondición:** Tener un tour registrado o en proceso de registro.
- **Flujo principal:** El administrador registra y organiza las actividades del itinerario, indicando el día, orden, horario, descripción y lugar.
- **Flujo alternativo:** Si una actividad requiere servicios de terceros, se la relaciona con el proveedor correspondiente.

25. Gestionar tarifas

- **Actor:** Administrador.
- **Precondición:** Tener un tour registrado.
- **Flujo principal:** El administrador registra las tarifas del tour según la temporada baja, media o alta, indicando el precio base y la fecha de vigencia.
- **Flujo alternativo:** No se especifica en el enunciado.

26. Gestionar proveedores

- **Actor:** Administrador.
- **Precondición:** Contar con los datos del proveedor.
- **Flujo principal:** El administrador registra los proveedores de la agencia, incluyendo nombre, CUIT, contacto y servicio que prestan.
- **Flujo alternativo:** No se especifica en el enunciado.

27. Liquidar honorarios de guías

- **Actor:** Administrador.
- **Precondición:** Tener una salida finalizada.
- **Flujo principal:** El sistema calcula los honorarios del guía a partir del monto fijo por día trabajado y el 5 % de las ventas de servicios adicionales de la salida.
- **Flujo alternativo:** Si la salida no está finalizada, no corresponde realizar la liquidación.

28. Registrar notas de ajuste

- **Actor:** Administrador.
- **Precondición:** Tener una reserva pagada que requiera una corrección.
- **Flujo principal:** El administrador registra una nota de ajuste para corregir la información necesaria, conservando el registro original.
- **Flujo alternativo:** No se permite eliminar una reserva pagada para realizar la corrección.

29. Emitir reporte de ocupación

- **Actor:** Administrador.
- **Precondición:** Tener salidas registradas.
- **Flujo principal:** El administrador solicita el reporte y el sistema muestra la ocupación correspondiente a cada salida.
- **Flujo alternativo:** No se especifica en el enunciado.

30. Emitir reporte de ventas

- **Actor:** Administrador.
- **Precondición:** Tener reservas asociadas a vendedores.
- **Flujo principal:** El sistema genera un reporte de ventas organizado por vendedor.
- **Flujo alternativo:** No se especifica en el enunciado.

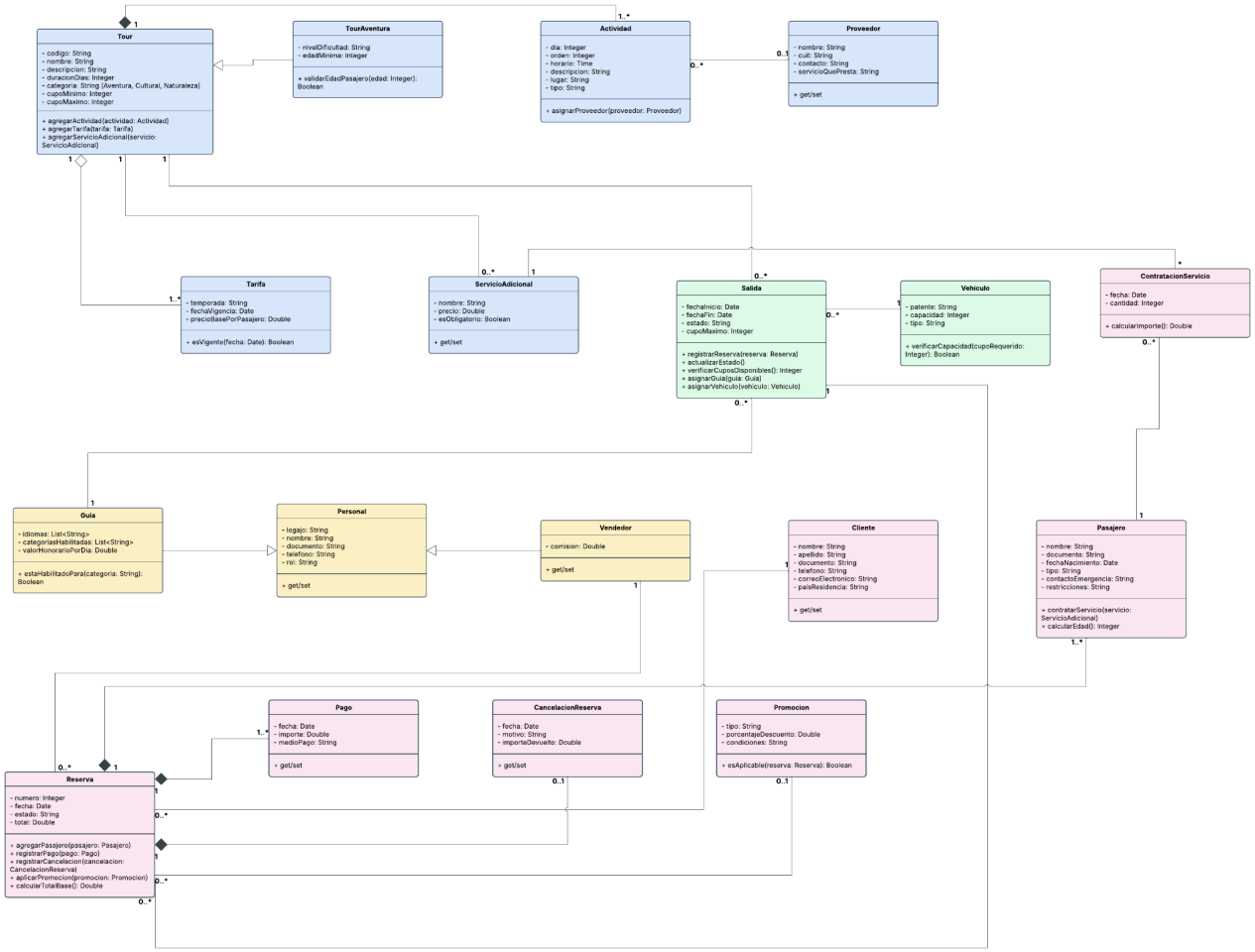
31. Emitir reporte de ingresos

- **Actor:** Administrador.
- **Precondición:** Tener información de las reservas y sus pagos.
- **Flujo principal:** El sistema genera un reporte de ingresos por tour.
- **Flujo alternativo:** No se especifica en el enunciado.

32. Emitir reporte de cancelaciones

- **Actor:** Administrador.
- **Precondición:** Tener cancelaciones registradas.
- **Flujo principal:** El sistema genera un reporte con la información de las reservas canceladas.
- **Flujo alternativo:** No se especifica en el enunciado.

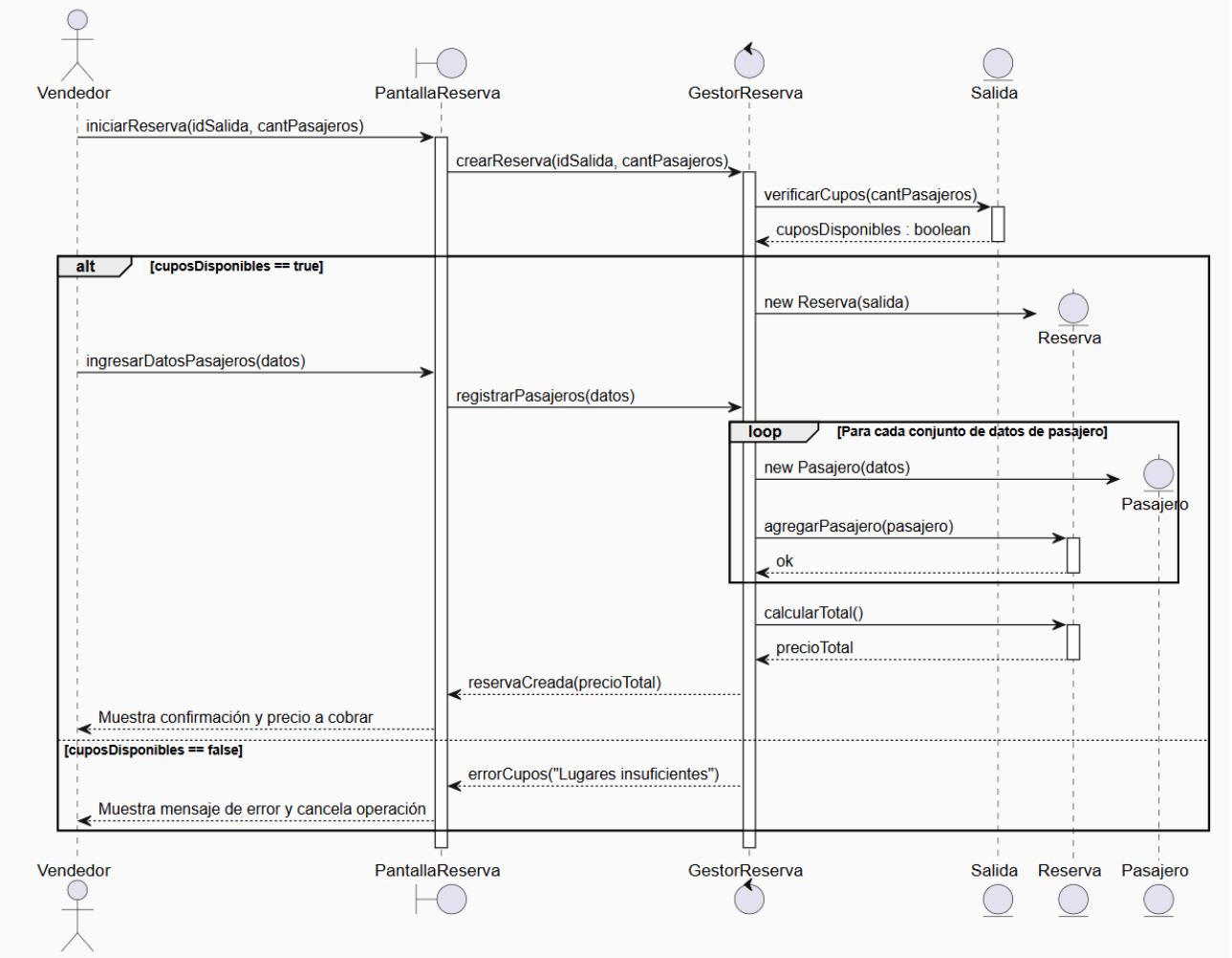
3. Diagrama de Clases Inicial



[LINK AL DIAGRAMA PARA MEJOR VISUALIZACIÓN]

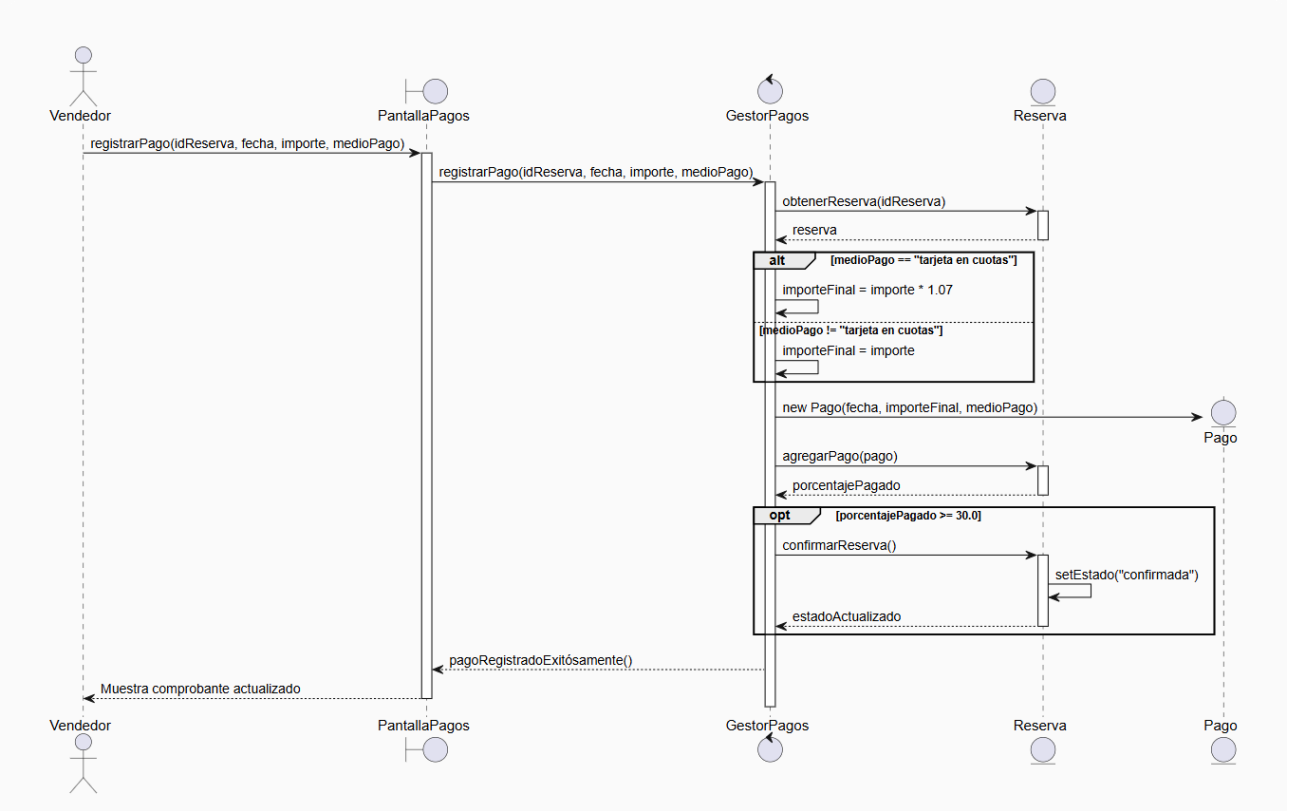
4. Diagramas de Secuencia

Caso de Uso 2: Crear Reserva



Este caso de uso tiene como flujo principal la creación de una reserva por parte del actor “Vendedor”. Esto conlleva un flujo alternativo cuando no hay cupos disponibles en la reserva para más pasajeros.
Además, tiene un loop para los posibles registros de múltiples pasajeros.

Caso de Uso 7: Registrar Pagos



Este caso de uso describe el registro de un pago sobre una reserva. Este caso cuenta con un bloque de flujo alternativo para cuando se abona con tarjeta en cuotas o no. Y también cuenta con un flujo opcional que hace una confirmación automática de la reserva cuando se alcanza la seña del 30%.

5. Aplicación de patrones

5.1. Principios SOLID

1. SRP — Principio de responsabilidad única

Problema detectado: En el diseño inicial, la clase **Reserva** concentraba distintas responsabilidades: administrar los pasajeros y el estado de la reserva, calcular su precio, aplicar promociones y gestionar la lógica económica de las cancelaciones. Esto generaba una clase con demasiadas responsabilidades y dificultaba modificar una parte del sistema sin afectar a las demás.

Decisión tomada: Se decidió separar las responsabilidades relacionadas con los cálculos económicos. Se incorporó la clase **CalculadoraPrecioReserva**, encargada de calcular el precio final de la reserva, considerando las tarifas, los pasajeros, los servicios adicionales y las promociones.

Además, se incorporó la interfaz **PoliticaCancelacion**, con sus implementaciones **PoliticaCancelacionCliente** y **PoliticaCancelacionAgencia**, para calcular las devoluciones según el motivo de la cancelación.

La clase **Reserva** conserva la responsabilidad de administrar los datos, los pasajeros y el estado de la reserva, sin concentrar toda la lógica económica.

Clases involucradas: **Reserva, CalculadoraPrecioReserva, PoliticaCancelacion, PoliticaCancelacionCliente y PoliticaCancelacionAgencia**.

Ventaja: Cada clase tiene una responsabilidad más clara. Esto facilita el mantenimiento y permite modificar las reglas de precios o cancelaciones sin tener que modificar toda la lógica de **Reserva**

2. OCP — Principio abierto/cerrado

Problema detectado: En el diseño inicial, la clase **Promocion** utilizaba un atributo tipo y un método **esAplicable()** para distinguir las promociones. Al incorporar una nueva promoción, era necesario agregar condiciones y modificar la lógica existente.

Decisión tomada: Se decidió introducir la interfaz **EstrategiaPromocion**, que define el método **calcularDescuento(Reserva)**. Las clases **PromocionAnticipada** y **PromocionGrupo** implementan esa interfaz y calculan sus respectivos descuentos.

La clase **CalculadoraPrecioReserva** utiliza estas estrategias para determinar qué promoción corresponde aplicar. Si existen varias promociones aplicables, selecciona la más beneficiosa, respetando la regla de negocio de que los descuentos no son acumulables.

Clases involucradas: **EstrategiaPromocion, PromocionAnticipada, PromocionGrupo y CalculadoraPrecioReserva.**

Ventaja: Es posible incorporar nuevas promociones creando nuevas implementaciones de **EstrategiaPromocion**, sin tener que modificar la estructura principal del cálculo de precios. Esto facilita la extensión del sistema y reduce el riesgo de introducir errores en funcionalidades existentes.

3. DIP — Principio de inversión de dependencias

Problema detectado: En el diseño inicial, la clase Pago representaba el medio de pago mediante un atributo de tipo **String**. Esto dificultaba encapsular las particularidades de cada medio de pago y podía llevar a incorporar condiciones específicas de efectivo, transferencia o tarjeta dentro de la lógica general del pago.

Decisión tomada: Se incorporó la interfaz **MedioPago**, que define el método **calcularMontoFinal(montoBase, cuotas)**. Las clases **Efectivo, Transferencia y TarjetaCredito** implementan esta interfaz y resuelven el cálculo según las características de cada medio. En particular, **TarjetaCredito** contempla el recargo del 7 % cuando se paga en cuotas.

La lógica que utiliza el medio de pago debe depender de la abstracción **MedioPago**, en lugar de quedar acoplada a una implementación concreta.

Clases involucradas: **Pago, MedioPago, Efectivo, Transferencia y TarjetaCredito.**

Ventaja: Se pueden incorporar o modificar medios de pago sin tener que modificar la lógica general que los utiliza. Esto reduce el acoplamiento y hace que el diseño sea más flexible.

5.2. Patrones GRASP

1. Experto en Información (Information Expert)

Problema detectado: Algunas operaciones del sistema necesitan información distribuida entre distintas clases. Si se asignaran todos los cálculos a una clase externa, esta tendría que consultar continuamente a otros objetos para obtener los datos necesarios.

Decisión tomada: Se asignaron las responsabilidades de cálculo a las clases que poseen la información necesaria para realizarlas. La clase **Salida** incorpora el método **calcularTotalAdicionalesVendidos()**, que obtiene el total vendido en servicios adicionales para esa salida. Por su parte, la clase **Guia** incorpora el método **calcularHonorarios(Salida)**, que calcula sus honorarios considerando los días trabajados y el porcentaje correspondiente sobre los servicios adicionales vendidos.

Clases involucradas: **Salida y Guia**.

Ventaja: La lógica queda ubicada cerca de los datos que necesita. Esto mejora la cohesión de las clases y evita distribuir innecesariamente los cálculos por el sistema.

2. Creador (Creator)

Problema detectado: Era necesario definir qué clase debía encargarse de crear los objetos **Pago** y **ContratacionServicio**, evitando que la responsabilidad de creación quede asignada arbitrariamente a una clase que no estuviera relacionada con ellos.

Decisión tomada: Se estableció que **Reserva** sea responsable de crear y registrar los objetos **Pago**, ya que administra los pagos correspondientes a la reserva. A su vez, **Pasajero** se encarga de crear y administrar sus objetos **ContratacionServicio**, porque los servicios adicionales se contratan individualmente por pasajero.

Clases involucradas: **Reserva, Pago, Pasajero y ContratacionServicio**.

Ventaja: Las responsabilidades de creación quedan asignadas a clases que mantienen una relación directa con los objetos creados. Esto simplifica la organización del sistema y facilita el mantenimiento de las relaciones entre los objetos.

3. Controlador (Controller)

Problema detectado: Si las solicitudes de los usuarios fueran procesadas directamente por las clases del dominio, se mezclaría la coordinación de las operaciones del sistema con las reglas propias del negocio.

Decisión tomada: Se incorporaron tres controladores que coordinan las operaciones principales del sistema:

- **ControladorVentas**: coordina las operaciones de registro de clientes, reservas, pasajeros, promociones y pagos.
- **ControladorOperaciones**: coordina la programación de salidas, la asignación de guías y vehículos, y el control de los cupos.
- **ControladorAdministrativo**: coordina las cancelaciones, las devoluciones, los honorarios y la generación de reportes.

Los controladores reciben las solicitudes de los actores internos y delegan las tareas en las clases correspondientes, sin concentrar en ellos las reglas de negocio.

Clases involucradas: **ControladorVentas, ControladorOperaciones, ControladorAdministrativo** y las clases del dominio con las que interactúan.

Ventaja: Se separa la coordinación de los casos de uso de la lógica del negocio. Esto mejora la organización, reduce el acoplamiento entre las distintas partes del sistema y facilita futuras modificaciones.

4. Fabricación pura (Pure Fabrication)

Problema detectado: La persistencia de la información en archivos no corresponde directamente a una responsabilidad propia de las entidades del negocio. Incorporar esa lógica dentro de clases como **Cliente, Tour o Reserva** aumentaría su complejidad y las acoplaría al mecanismo de almacenamiento.

Decisión tomada: Se incorporó la interfaz genérica **IRepository<T>**, que define las operaciones básicas de persistencia. Se proponen implementaciones específicas como **ClienteRepository, TourRepository, ReservaRepository y SalidaRepository**, responsables de gestionar el almacenamiento y la recuperación de sus respectivas entidades.

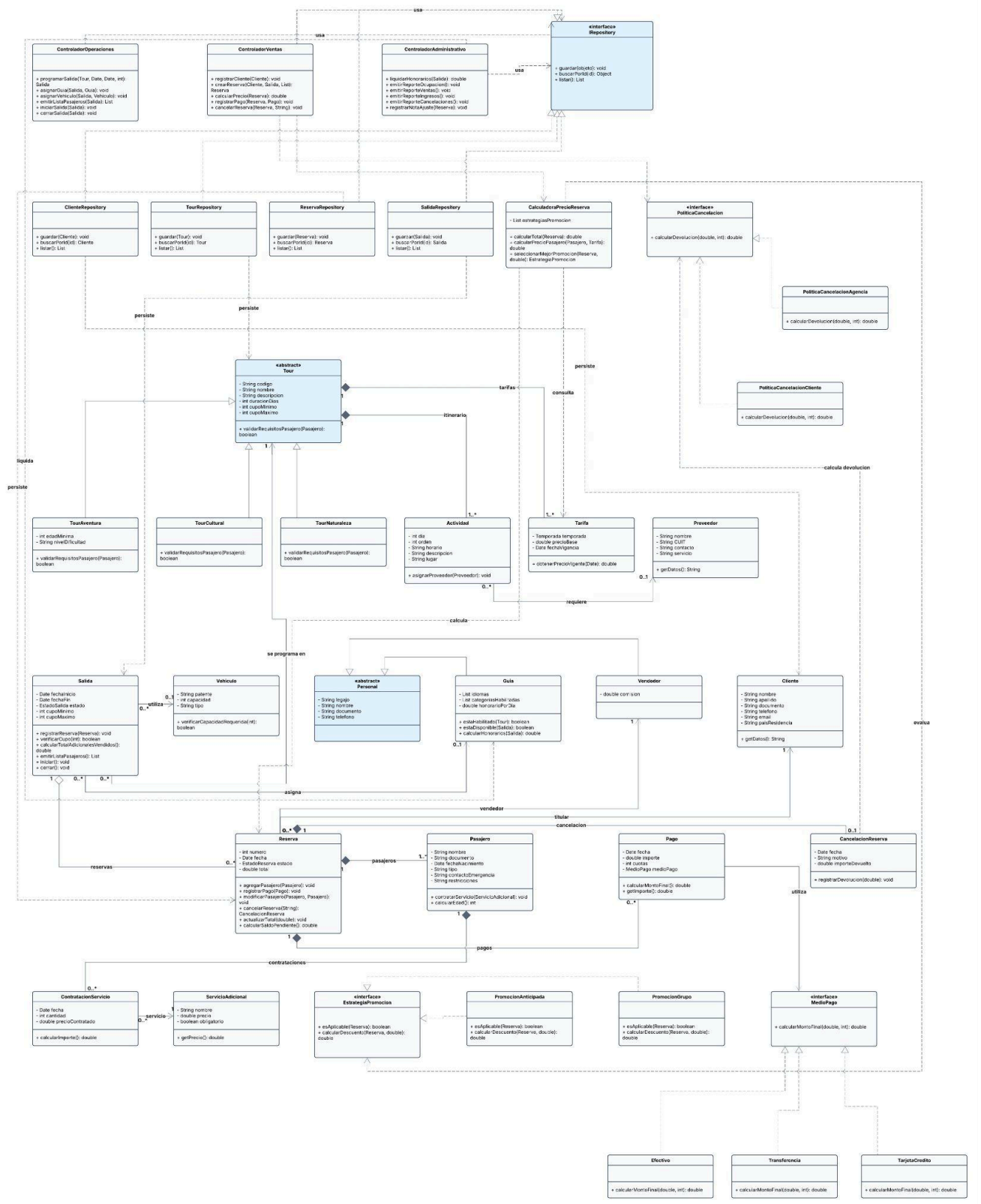
Los controladores utilizan las operaciones de persistencia a través de la abstracción correspondiente, sin tener que conocer los detalles internos del almacenamiento.

Clases involucradas: **IRepository<T>, ClienteRepository, TourRepository, ReservaRepository, SalidaRepository** y los controladores.

Ventaja: Se separa la persistencia de la lógica del negocio. Esto facilita modificar el mecanismo de almacenamiento, reutilizar las operaciones de persistencia y mantener las entidades del dominio independientes de los archivos.

6. Diagrama de clases final

Se presenta el diagrama de clases final de la agencia de turismo Rumbo Sur, incorporando las modificaciones derivadas de los principios SOLID y los patrones GRASP aplicados.



LINK AL DIAGRAMA PARA MEJOR VISUALIZACIÓN

Cambios realizados respecto del diagrama de clases inicial

A partir de la revisión del diseño inicial, se realizaron modificaciones para mejorar la distribución de responsabilidades, reducir el acoplamiento y facilitar la extensión del sistema.

1. Separación de responsabilidades (SRP). Se incorporó la clase **CalculadoraPrecioReserva** para centralizar el cálculo del precio final de las reservas. También se agregó la interfaz **PoliticaCancelacion**, junto con **PoliticaCancelacionCliente** y **PoliticaCancelacionAgencia**, para separar la lógica de las devoluciones de la administración general de una reserva.

2. Incorporación de estrategias de promoción (OCP). La clase **Promocion** fue reemplazada por la interfaz **EstrategiaPromocion** y las clases **PromocionAnticipada** y **PromocionGrupo**. De esta manera, se pueden agregar nuevas promociones sin modificar la estructura principal del cálculo de precios.

3. Abstracción de los medios de pago (DIP). Se incorporó la interfaz **MedioPago**, implementada por **Efectivo, Transferencia y TarjetaCredito**. Esto permite representar las particularidades de cada medio de pago de manera independiente, incluyendo el recargo correspondiente a los pagos con tarjeta en cuotas.

4. Aplicación de Experto en Información. Se asignaron a **Salida y Guia** los métodos **calcularTotalAdicionalesVendidos() y calcularHonorarios(Salida)**, respectivamente, ubicando los cálculos en las clases que cuentan con la información necesaria.

5. Aplicación de Creador. Se estableció que **Reserva** cree y registre los objetos **Pago**, mientras que **Pasajero** cree y administre los objetos **ContratacionServicio**, de acuerdo con las relaciones que mantienen con ellos.

6. Incorporación de controladores. Se agregaron **ControladorVentas**, **ControladorOperaciones y ControladorAdministrativo** para coordinar los principales casos de uso y delegar las operaciones en las clases correspondientes.

7. Separación de la persistencia (Fabricación pura). Se incorporó la interfaz genérica **IRepository<T>** y sus implementaciones **ClienteRepository, TourRepository, ReservaRepository y SalidaRepository**, para separar el almacenamiento de la lógica del negocio.

En conjunto, estos cambios permiten obtener un diseño con responsabilidades mejor distribuidas, menor acoplamiento entre componentes y mayor facilidad para mantener y ampliar el sistema.